SPG Integration Document

Version 2.0

November 2022

Contents

Overview **3**

Introduction **3**

High-Level Integration Diagram **4**

Sandbox and Production Environment **5**

SPG Domains 5

Credentials 5

Processing orders using SPG checkout **6**

Payment Flow 6

Transaction Flow 6

APIs to be integrated 6

Integration Steps 6

API Documentation **7**

Checkout 8

Transaction Status 12

Transaction Refund 15

Signature Logic **17**

Appendix **20**

Security 20

HTTP Codes 20

Response Codes 21

PHP Code Snippets 24

Overview

This document describes the steps for technical integration process between merchant website and SPG to allows merchants to instantly collect payments from their users using SPG checkout.

Introduction

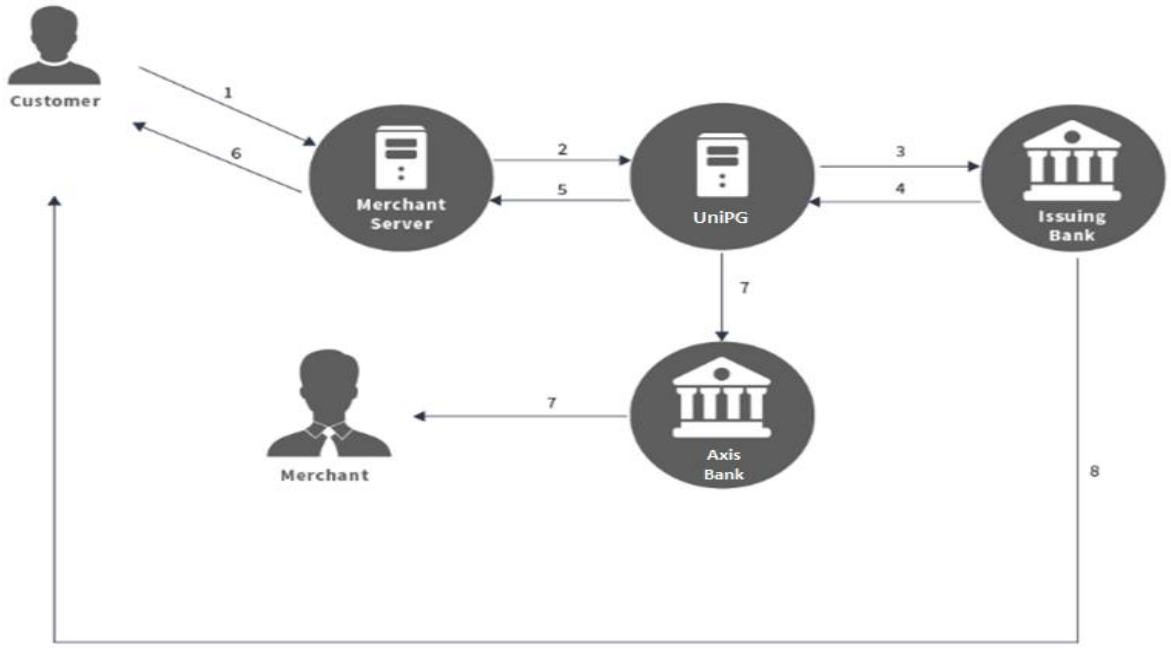
SPG offers a flexible set of APIs for processing electronic commerce payments through SPG platform. Connect your application directly to the SPG payment platform to process a full suite of methods that are REST-based – designed to be easy to understand and use.

Through SPG, the customers would be able to make electronic payments through a variety of modes which are mentioned below:

- Credit cards
- Debit cards
- Netbanking
- UPI

SPG also offers an online portal to access transaction reporting, refund initiation and account management features. This online interface can be accessed through the username and password provided to you.

High-Level Integration Diagram



1. Customer selects product/service on merchant website and proceeds to make payment.
2. Merchant makes a checkout payment request to SPG. (Browser Redirect)
3. SPG validates the request and renders the payment checkout page on the customer's browser.
4. The customer chooses his desired mode of payment, furnishes the payment details, and clicks on Pay.
5. SPG initiates the payment and routes the transaction to the customer's bank for authentication where the customer enters OTP/PIN.
6. SPG receives the transaction authentication response from Bank.
7. SPG authorises the transaction with the bank.
8. SPG sends the final transaction response to merchant. (Browser Redirect)
9. Merchant parses the response and displays the relevant view to the customer and proceeds with its processing.
10. On subsequent day, SPG will settle the transaction amount to merchant along with the settlement file.
11. Customer's bank will furnish the transaction details in customer account statement.

Sandbox and Production Environment

SPG test and production environments are separate.

Merchants need an active account to use the test and production environment.

Test environment is strictly for testing the request and response functions. After successfully testing the integration, merchant can move to the production environment by changing the URL.

Credentials

Following credentials are required to test the integration:

1. Merchant ID
2. Merchant Key

Domains

Sandbox: https://sandbox-axispg.freecharge.in
Production: https://secure-axispg.freecharge.in

Merchant ID and Key is shared over welcome email.

Processing orders using SPG checkout:

Payment flow

1. Customer selects product/service on merchant website and proceeds to make payment.
2. Customer is redirected to the SPG checkout where payment information is entered by the customer.
3. On submission of the transaction information, SPG redirects the customer to the Bank's authentication page.
4. On transaction completion, SPG redirects the customer back to the merchant website along with the transaction response.

Transaction Flow

1. To initiate a transaction, the merchant needs to redirect its customer to the SPG checkout endpoint through an HTTPS POST Request.
2. On the SPG checkout page the customer would then select the payment method and click on the 'Pay Now' button. SPG would then re-direct the customer to the chosen bank. The bank would then redirect the customer back to SPG post authentication.
3. SPG would then redirect the customer back to the merchant post authorisation, if required, on the call-back URL specified in the initial checkout redirect request through an HTTPS Post Request. Response parameters are covered in the subsequent sections.
4. In the response sent by SPG, you would receive the final status of the transaction. It is necessary to verify the signature in this response at your end and only then to accept/reject the transaction. This is to avoid tampering attempts in the transaction.

APIs to be integrated:

1. Checkout
2. Transaction Refund
3. Transaction Status

Integration Steps

1. Be ready with the Prerequisite as mentioned in welcome email.
2. Prepare the Signature as mentioned in the Signature section using the parameters and merchant id and Merchant Key.
3. Define the Call-back URL
4. Integrate with Checkout API as per API documentation section.
5. Post the "transaction request parameters" on the API URL
6. Accept the "transaction response parameter" from SPG on call-back URL after completion of transaction.

API Documentation

1. Checkout

- Method: POST Redirect
- Accepts: application/x-www-form-urlencoded
- Returns: View/HTML
- URL: {SPG Domain}/payment/v1/checkout

1.1 Request Parameters

Parameter	Data Type	Mandatory	Description
merchantId	String	Yes	Unique merchant identifier provided by SPG Example – MERMER991z11a
callbackUrl	String	Yes	Merchant endpoint to which customer would be redirected post transaction. Example – https://merchanturl.com/cb
merchantTxnId	String	Yes	Unique transaction identifier sent by the merchant. Example – SH2397807631
merchantTxnAmount	Double	Yes	Transaction Amount Example – 3275.80
Currency	Fixed Value/ String	Yes	Fixed Value - INR
tags	String	No	For future use
customerId	String	No	Merchant customer ID Example - C0Ds8q
customerName	String	No	Customer Name Example – 'John Doe'
customerEmailId	String	No	Customer Email Address Example – 'jd7383@mac.com'
customerMobileNo	String	No	Customer Mobile Number with Country Code Example – '919199859854'
customerStreetAddress	String	No	Customer Street Address Example – 'DLF, Ph - 3'
customerCity	String	No	Customer City

			Example – 'Gurgaon'
customerState	String	No	Customer State Example – 'Haryana'
customerPIN	String	No	Customer Pin Example – '122001'
customerCountry	String	No	Customer Country Example – 'India'
timestamp	Long	Yes	EPOCH, Current timestamp Example - 1587481920
Signature	String	Yes	
subMerchantPayInfo	String (Json Format)	No	To be sent in case merchant wants to split his payment into multiple sub-merchant payments and direct sub-merchant settlements. (Table 1.1.1)
udf1	String	No	Parameters for additional fields to be captured in payment request
udf2	String	No	Same as udf1
udf3	String	No	Same as udf1
udf4	String	No	Same as udf1
udf5	String	No	Same as udf1

1.1.1 SubMerchantPayInfo

Parameter	Data Type	Mandatory	Description
subEntityId	String	Yes	Identifier provided by SPG for split payment beneficiary entities Example – MERSP991z12
subEntityTxnAmount	Double	Yes	Split Payment Amount Example – 200.40
subEntityType	Integer	No	Entity Type
subEntityProduct	String	Yes	Product identifier for split transaction

If merchantTxnID and merchantID combination are repeated, the request will that request and respond with an appropriate statusCode. The merchant should use the txnStatus API to fetch the status of that merchantTxnId.

Sample Request

```
curl -X POST
http://{SPGDomain}/payment/v1/checkout
-H 'cache-control: no-cache'
-H 'content-type: application/x-www-form-urlencoded'
-H 'postman-token: ba9b9dfa-6efa-4d59-1db1-0462930f74d1'
-d 'merchantId=merchantId001
&callbackUrl=http://{MerchantAPIURL}
&merchantTxnId=merchantTxn001
&merchantTxnAmount=1000.0
&currency=INR
&tags=
&customerId=customerId001
&customerName=Aakash
&customerEmailId=akash@gmail.com
&customerMobileNo=919199859854
&customerStreetAddress=DLF Ph 3
&customerCity=GURGAON
&customerState=Haryana
&customerPIN=122002
&customerCountry=India
&timestamp=1646649292
&subMerchantPayInfo=[{"subEntityId": "entity1", "subEntityTxnAmount": "500.0",
   "subEntityProduct": "P1"},{"subEntityId": "entity2","subEntityTxnAmount":
   "500.0", "subEntityProduct": "P2"}]
&udf1=Policy 123
&udf2=Docket 1
&udf3=ETB
&udf4=Small
&udf5=Priority
&signature=8393065041bec186972615c583b86d2c60d87a1ec22f88dba5a7de594b1649b8'
```

1.2 Payment Response Call-back

This is the final payment response from SPG. SPG redirects the customer back to the merchant post the transaction and the parameters in the call-back as given below. Merchant needs to parse this response and needs to take further actions based on the status returned.

1.2.1 Response Parameters

Method: POST Redirect
Produces: application/x-www-form-urlencoded

Parameter	Data Type	Mandatory	Description
statusCode	String	Yes	Transaction Status Code Example – SPG-0000 (Also check Response Code Table below)
statusMessage	String	Yes	Transaction Status Message Example – 'Success'
merchantTxnId	String	Yes	SPG unique transaction identifier Example – US6284719503
txnReferenceId	String	Yes	Required for authenticity of the data. Example – US6284719503
amount	Decimal	Yes	Transaction Amount Example – 3275.80
Currency	String	Yes	Transaction Currency Value – INR
handlingFee	Decimal	Yes	Convenience Fee Charged on Transaction Example – 65.30
taxAmount	Decimal	Yes	Tax applied on Convenience Fee. Example – 11.80
Mode	String	Yes	Payment mode. Example: DC / CC / NB
Submode	String	Yes	Sub-mode VI/MC etc for cards Bank code in case of NB
issuerCode	String	No	Issuer code. Example: 0004
issuerName	String	No	Card Issuer Name. Example: Axis Bank
Signature	String	Yes	Checksum Verification
subMerchantPayInfo	String JSON	No	Split transaction details for each entity and product (Refer Table 1.2.2)

1.2.2 SubMerchantPayInfo

Parameter	Data Type	Mandatory	Description
subEntityId	String	Yes	Identifier provided by SPG for split payment beneficiary entities
subEntityTxnAmount	Double	Yes	Split Payment Amount Example – 200.40
subEntityProduct	String	Yes	Product identifier for split transaction
subTransactionId		Yes	Unique Transaction ID generated by SPG for each split payment
subTxnSurchargeInfo		Yes	Only if Surcharge is enabled (Table 1.2.3)

1.2.3 SubMerchantPayInfo

Parameter	Data Type	Mandatory	Description
handlingFee	String	Yes	Convenience Fee Charged on Transaction
taxAmount	Double	Yes	Tax applied on Convenience Fee.

Sample Request

```
1. curl -X POST
2. https://{MerchantAPIURL}
3. -H 'cache-control: no-cache'
4. -H 'content-type: application/x-www-form-urlencoded'
5. -H 'postman-token: dc90b214-b54b-2ae0-f61f-f1675d09cdfc'
6. -d 'statusCode=SPG-0000
7. &statusMessage=SUCCESS
8. &merchantTxnId=MerTxnId23Feb001
9. &txnReferenceId=15e739109336571
10. &amount=1000.0
11. &currency=INR
12. &handlingFee=1.0
13. &taxAmount=1.0
14. &tags=
15. &mode=CC
16. &subMode=VI
17. &issuerCode=
18. &issuerName=
19. &signature=2db5c357e3c489cea3e3e783107d275e982ea25beefd39344ad8b74efad53b02
20. &subMerchantPayInfo=[{"subEntityId":"MERMERbd5b001","subEntityTxnAmount":"500.
    0","subEntityProduct":"P1","subTransactionId":"d4ec3a636337043","subTxnSurchar
    geInfo":{"handlingFee":0.0,"taxAmount":0.0}},{"subEntityId":"MERMERbd5b9d7","s
    ubEntityTxnAmount":"500.0","subEntityProduct":"P2","subTransactionId":"d4ec3a6
    3222222","subTxnSurchargeInfo":{"handlingFee":0.0,"taxAmount":0.0}}]
```

2. Transaction Status

- Method: POST
- Accepts: JSON
- Returns: JSON
- Endpoint: {SPG Domain}/payment/v1/txn/status

2.1 Request Parameters

Headers: 'Content-Type : application/json'

Parameter	Data Type	Mandatory	Description
merchantId	String	Yes	Unique merchant identifier provided by SPG. Example – MERMER991z11a
merchantTxnId	String	Yes	Unique transaction identifier sent by the merchant/aggregator. Example – SH2397807631
txnreferenceId	String	No	Unique transaction identifier generated by SPG for Pay or Refund transaction Example – US6284719503
signature	String	Yes	Required for authenticity of the data.

2.2 Response Parameters

Parameter	Data Type	Mandatory	Description
statusCode	String	Yes	Transaction Status Code Example – SPG-0000 (Also check Response Code Table below)
statusMessage	String	Yes	Transaction Status Message Example – 'Success'
data	String	Yes	Refer Table 2.2.1

2.2.1 Parameters in Data

Parameter	Data Type	Mandatory	Description
merchantTxnId	String	Yes	Unique transaction identifier sent by the merchant. Example – SH2397807631
txnReferenceId	String	Yes	SPG unique transaction identifier Example – US6284719503
txnType	String	Yes	Type of transaction Example - Pay/Refund
amount	Decimal	Yes	Transaction Amount Example – 3275.80
currency	String	Yes	Transaction Currency Value – INR
Tags	String	NA	For future use
merchantId	String	No	
createdDate	String	No	
surchargeInfo	Surcharge Info	NA	Refer Table 2.2.1.1

2.2.1.1 Surcharge Info – Applicable for few merchants only

Parameter	Data Type	Max Length	Description
handlingFee	Decimal	NA	Convenience Fee Charged on Transaction Example – 65.30
taxAmount	Decimal	NA	Tax applied on Convenience Fee. Example – 11.80

Sample Request

```
1. {
2.   "merchantId" : "MERMER991z11a",
3.   “merchantTxnId” : “SH2397807631”,
4.   “txnreferenceId” : “US6284719503",
5.   “signature” : “8393065041bec186972615c583b86c60d87ac22f885a7de1649b8”
6. }
```

Sample Response

```
1. {
2.   "statusCode": "SPG-0000",
3.   "statusMessage": "SUCCESS",
4.   "data":
5.             {
6.             "merchantTxnId": "SH2397807631",
7.             "txnReferenceId": "US6284719503",
8.             "amount": 3275.80,
9.             "currency": "INR",
10.                  "surchargeInfo":
11.                                   {
12.                                    “handlingFee” : 65.30,
13.                                    “taxAmount”: 11.80
14.                                    }
15.                  "tags": null,
16.                  "txnType": "PAY"
17.                   }
18.      }
```

3. Transaction Refund

- Method: POST Redirect
- Accepts: JSON
- Returns: JSON
- Endpoint:
  - Sandbox**:** {SPG Domain}/payment/v1/refund
  - Production**:** {SPG Domain}/payment/v1/refund

3.1 Request Parameters

Headers: 'Content-Type : application/json'

Parameter	Data Type	Mandatory	Description
merchantId	String	Yes	Unique merchant identifier provided by SPG. Example – MERMER991z11a
merchantRefundTxnId	String	Yes	Unique transaction identifier sent by the merchant. Example – SH2397807631
txnReferenceId	String	Yes	Unique transaction identifier generated by SPG for Pay transaction. *This will be subTransactionId in case of split payments* Example – US6284719503
refundAmount	Double	Yes	Amount to be refunded. *This will be Refund amount against subTransactionId in case of split payments* Example – 50
Currency	Fixed Value/ String	Yes	Fixed Value - INR
Signature	String	Yes	Required for authenticity of the data

3.2 Response Parameters

Headers: 'Content-Type : application/json'

Parameter	DataType	Mandatory	Description
statusCode	String	Yes	Transaction Status Code Example – SPG-0000 (Also check Response Code Table below)
statusMessage	String	Yes	Transaction Status Message Example – 'Success'
Data	String	Yes	Refer Table 3.2.1

3.2.1 Parameters in Data

Parameter	DataType	Mandatory	Description
merchantTxnId	String	Yes	Unique transaction identifier sent by the merchant. Example – SH2397807631
txnReferenceId	String	Yes	SPG unique transaction identifier Example – US6284719503
refundAmount	Double	Yes	Amount to be refunded. Example – 50

Sample Request

```
1. {
2.   “merchantId” : “MERMER991z11a”,
3.   “merchantTxnId” : “SH2397807631”,
4.   "txnReferenceId" : "US6284719503",
5.   “refundAmount” : 50,
6.   “currency” : “INR”,
7.   “signature” : “8393065041bec186972615c583b86d2c60d875a7de594b1649b8”
8. }
```

Sample Response

```
1. {
2.   "statusCode": "SPG-0000",
3.   "statusMessage": "SUCCESS",
4.   "data": {
5.               "merchantTxnId": "SH2397807631",
6.               "txnReferenceId": "US6284719503",
7.               "refundAmount": 50.0
8.        }
9. }
```

Signature Logic

All requests sent to SPG need to have a signature using which SPG will identify the authenticity of the request. Follow the below steps to create the signature for any API request.

1. Create the request with all the required fields except the signature.
2. Concatenate all the non-null field values (required in the signature) in ascending order of the field names.
3. Concatenate the Merchant Key (provided to you during onboarding) to the concatenated string generated in step 2.
4. Create a hash of the string generated in step 3 using the algorithm SHA-256.
5. Hex Encode the resultant bye stream generated in step 4.
6. Hex Encoded string generated in step 5 is your signature.

Signature Generation Walkthrough (Java)

Let's say you are creating the request for the Merchant Checkout API. (The same process works for any other API that you integrate with)

Step 1 Create the request.

```
MerchantCheckoutPayRequest obj = new MerchantCheckoutPayRequest();
        obj.setMerchantId("fieldValue1");
        obj.setMerchantTxnAmount(fieldValue2);
        obj.setMerchantTxnId("fieldValue3");
        obj.setCallbackUrl("fieldValue4");
        obj.setCurrency("fieldValue5");
        obj.setCustomerId("fieldValue6");
        obj.setCustomerName("fieldValue7");
        obj.setTxnMode("fieldValue8");
        obj.setTxnSubMode("fieldValue9");
        obj.setTimestamp(fieldValue10);
```

Step 2

To concatenate all the field values in ascending order of the field names you can use a TreeMap in Java. Put all the field name -value pairs in the map. Iterate the map and create the concatenated data to hash.

```
Map<String, String> tmpobject = new TreeMap<String, String>();
tmpobject.put("merchantId", String.valueOf(objMerchantCkOutPay.getMerchantId()));
tmpobject.put("callbackUrl", String.valueOf(objMerchantCkOutPay.getCallbackUrl()));
tmpobject.put("customerId", String.valueOf(objMerchantCkOutPay.getCustomerId()));
tmpobject.put("customerName", String.valueOf(objMerchantCkOutPay.getCustomerName()));
tmpobject.put("customerEmailId", String.valueOf(objMerchantCkOutPay.getCustomerEmailId()));
tmpobject.put("customerMobileNo",
String.valueOf(objMerchantCkOutPay.getCustomerMobileNo()));
tmpobject.put("customerStreetAddress",
String.valueOf(objMerchantCkOutPay.getCustomerStreetAddress()));
tmpobject.put("customerCity", String.valueOf(objMerchantCkOutPay.getCustomerCity()));
tmpobject.put("customerState", String.valueOf(objMerchantCkOutPay.getCustomerState()));
tmpobject.put("customerPIN", String.valueOf(objMerchantCkOutPay.getCustomerPIN()));
tmpobject.put("customerCountry", String.valueOf(objMerchantCkOutPay.getCustomerCountry()));
tmpobject.put("merchantTxnId", String.valueOf(objMerchantCkOutPay.getMerchantTxnId()));
tmpobject.put("merchantTxnAmount",
String.valueOf(objMerchantCkOutPay.getMerchantTxnAmount()));
tmpobject.put("currency", String.valueOf(objMerchantCkOutPay.getCurrency().code()));
tmpobject.put("txnMode", String.valueOf(objMerchantCkOutPay.getTxnMode().code()));
tmpobject.put("txnSubMode", String.valueOf(objMerchantCkOutPay.getTxnSubMode()));
tmpobject.put("cardHolderName", String.valueOf(objMerchantCkOutPay.getCardHolderName()));
tmpobject.put("cardNumber", String.valueOf(objMerchantCkOutPay.getCardNumber()));
tmpobject.put("cardExpiryMonth", String.valueOf(objMerchantCkOutPay.getCardExpiryMonth()));
tmpobject.put("cardExpiryYear", String.valueOf(objMerchantCkOutPay.getCardExpiryYear()));
tmpobject.put("cardCvv", String.valueOf(objMerchantCkOutPay.getCardCvv()));
tmpobject.put("timestamp", String.valueOf(objMerchantCkOutPay.getTimestamp()));

StringBuilder dataToHash = new StringBuilder();

for (Map.Entry<String, String> entry : tmpobject.entrySet()) {
  System.out.println(entry.getKey()+"  :: "+entry.getValue());
  if(null != entry.getValue()) {
    dataToHash.append(entry.getValue());
  }
}
        obj.setTimestamp(fieldValue10);
```

Step 3

Append the merchant key to the generated string.

```
dataToHash.append(merchantKey);
```

Step 4

Generate the hash of the dataToHash using SHA-256. We have provided a sample implementation of the SHA-256 algorithm below for merchants to use.

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;

public class HashCalculator {

  public static byte[] generateSHA256(String dataToHash) {
    String returnValue = null;
    try {
      MessageDigest md = MessageDigest.getInstance(CommonConstants.SHA);
      md.reset();
      md.update(dataToHash.getBytes(StandardCharsets.UTF_8));
      byte[] messageDigest = md.digest();


    } catch (Exception e) {
      throw new RuntimeException("Hash Generation Exception");
    }
    return returnValue;
  }
}
```

Step 5

Hex Encode the byte stream generated in step 5. We have provided a sample implementation of Hex Encoding in the code below for merchants to refer.

```
public class Encoder {

  public static String hexEncode(byte[] messageDigest) {
    StringBuilder hexString = new StringBuilder();
      for (int i = 0; i < messageDigest.length; i++) {
        String hex = Integer.toHexString(0xFF & messageDigest[i]);
        if (hex.length() == 1) {
          hexString.append("0");
        }
        hexString.append(hex);
      }
      returnValue = hexString.toString();
  }
}
```

Appendix

Security

TLS 1.2 , HTTPS, CKSUM , Encrypted Request and Response,

HTTP Codes

HTTP CODE	CATEGORY/STATUS	MESSAGE
200	REQUEST COMPLETED	Determine Response from Status Code
500	PENDING	Internal server error
400	FAILED	Invalid Request Data
401	FAILED	Unauthorized / authentication exception

Response Codes

STATUS CODE	CATEGORY/STATUS	MESSAGE
SPG-0000	SUCCESS	SUCCESS
SPG-0001	FAILED	FAILED
SPG-0002	PENDING	PENDING
SPG-0003	FAILED	Invalid merchant id
SPG-0004	FAILED	Signature is absent in the request
SPG-0005	FAILED	Invalid signature
SPG-0006	FAILED	Invalid session/ Session expired
SPG-5000	PENDING	Internal server error
SPG-0008	FAILED	Unable to process request
SPG-0009	FAILED	Transaction declined because of expired key. Please contact support
SPG-0010	FAILED	Invalid request field[s]
SPG-0011	PENDING	Socket Timeout
SPG-0014	FAILED	Duplicate request for merchant id and merchant txn id

SPG-0019	FAILED	Invalid payment mode
SPG-0025	FAILED	Transaction does not exist
SPG-0027	FAILED	Payment txn not present for refund request
SPG-0028	FAILED	Duplicate refund request. Kindly use txn status API
SPG-0029	FAILED	Refund amount has exceeded the payment amount
SPG-0030	FAILED	Invalid combination of txn reference id & merchant txn id
SPG-0032	PENDING	Error while making HTTP call
SPG-0033	FAILED	Invalid card expiry month
SPG-0034	FAILED	Invalid card expiry year
SPG-0035	FAILED	Invalid card cvv
SPG-0036	FAILED	Invalid card number
SPG-0037	FAILED	Card has expired
SPG-0038	PENDING	Client error
SPG-0040	PENDING	SSL setup error
SPG-0041	FAILED	Merchant not authorized for the API

SPG-0042	FAILED	Refund amount is more than pay amount
SPG-0044	FAILED	Merchant Id in request does not match that of in pay request
SPG-0028	FAILED	Duplicate authorization request
SPG-8000	PENDING	Payment authorization pending
SPG-8001	FAILED	Payment authorization failed
SPG-4000	FAILED	Bad request
SPG-4001	FAILED	Merchant Id does not match that of txn
SPG-5001	FAILED	Duplicate PG response
SPG-5002	FAILED	Incorrect combination of mode/submode
SPG-5003	FAILED	PG unavailable
SPG-5004	FAILED	Invalid card number entered
SPG-5005	FAILED	Invalid bank code
SPG-0048	FAILED	Pay and refund are in different environments;

PHP Code Snippets

Checkout

Step 1: Create PaymentCheckoutRequest

```
$paymentCheckoutRequest=new PaymentCheckoutRequest();
 $paymentCheckoutRequest ->setMerchantId( "MERMER93c38c3");
 $paymentCheckoutRequest ->setCallbackUrl( "https://google ->com");
 $paymentCheckoutRequest ->setMerchantTxnId( "");
 $paymentCheckoutRequest ->setMerchantTxnAmount( "10 ->0");
 $paymentCheckoutRequest ->setCurrency( "INR");
 $paymentCheckoutRequest ->setTags( "\"\"");
 $paymentCheckoutRequest ->setCustomerId( "customerId001");
 $paymentCheckoutRequest ->setCustomerName( "Aakash");
 $paymentCheckoutRequest ->setCustomerEmailId( "akash@gmail ->com");
 $paymentCheckoutRequest ->setCustomerMobileNo( "9191998598");
 $paymentCheckoutRequest ->setCustomerStreetAddress( "DLF Ph 3");
 $paymentCheckoutRequest ->setCustomerCity( "GURGAON");
 $paymentCheckoutRequest ->setCustomerState( "Haryana");
 $paymentCheckoutRequest ->setCustomerPIN("122002");
 $paymentCheckoutRequest ->setCustomerCountry( "India");
 $paymentCheckoutRequest ->setTimestamp("");
 $paymentCheckoutRequest ->setSubMerchantPayInfo("");
 $paymentCheckoutRequest ->setUdf1( "udf1");
 $paymentCheckoutRequest ->setUdf2( "udf2");
 $paymentCheckoutRequest ->setUdf3( "udf3");
 $paymentCheckoutRequest ->setUdf4( "udf4");
 $paymentCheckoutRequest ->setUdf5( "udf5");
 $paymentCheckoutRequest ->setSignature(
"424aef13477aed215c0d37e018322ae4d14dda1e1ee75887ee1299e18a6ffc26");
```

Step 2: Create a string from given data to be sent to api

```
$post_data="";
foreach ((array)$paymentCheckoutRequest as $key => $value) {
    $curr=str_replace("PaymentCheckoutRequest","",$key);
    $curr=ltrim($curr);
    $post_data.=($curr."=".$value."&");
}
$post_data=rtrim($post_data,"&");
```

Step 3: Use curl_init() to initialize a new session and return a cURL handle. (Used cURL service in Php)

```
$curl = curl_init();
```

Step 4: Specify all parameters required and change CURLOPT_HTTPHeader to array('Content-Type: application/x-www-form-urlencoded')

```
curl_setopt_array($curl, array(
        CURLOPT_URL => 'https://spg-paymentorchestrator-qa.fcinternal.in/payment/v1/initPay',
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_SSL_VERIFYPEER => false,
        CURLOPT_ENCODING => '',
        CURLOPT_MAXREDIRS => 10,
        CURLOPT_TIMEOUT => 0,
        CURLOPT_FOLLOWLOCATION => true,
        CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
        CURLOPT_CUSTOMREQUEST => 'POST',
        CURLOPT_POSTFIELDS => $json,
        CURLOPT_HTTPHEADER => array(
                'Content-Type: application/json'
        ),
));
```

Step 5: Execute the request .

```
$response = curl_exec($curl);
```

Refund

Step 1: Create RefundRequest

```
$refundRequest=new RefundRequest();
$refundRequest->setCurrency("INR");
$refundRequest->setMerchantId("MERMER34338c3");
$refundRequest->setMerchantRefundTxnId("16643597477483");
$refundRequest->setTimestamp("1668494373");
$refundRequest->setRefundAmount("1");
$refundRequest->setSignature("4f9fffgr10c6692320b808ae182ee6e27afd6617e5d310a7c5c18583ea04caad");
$refundRequest->setTxnReferenceId("jkr18c4ce2763063");
```

Step 2: Convert Request Object to Json . (Used Php Json Extension)

```
$json=json_encode($refundRequest);
```

Step 3: Use curl_init() to initialize a new session and return a cURL handle.(Used cURL service in Php)

```
$curl = curl_init();
```

Step 4: Specify all required

```
curl_setopt_array($curl, array(
        CURLOPT_URL => 'https://spg-paymentorchestrator-qa.fcinternal.in/payment/v1/refund',
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_SSL_VERIFYPEER => false,
        CURLOPT_ENCODING => '',
        CURLOPT_MAXREDIRS => 10,
        CURLOPT_TIMEOUT => 30000,
        CURLOPT_FOLLOWLOCATION => true,
        CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
        CURLOPT_CUSTOMREQUEST => 'POST',
        CURLOPT_POSTFIELDS => $json,
        CURLOPT_HTTPHEADER => array(
                'Content-Type: application/json'
        ),
));
```

Step 5: Execute the request

```
response = curl_exec($curl);
```

Step 6: Deserialize the received response to a class object. (Used Php Json Extension)

```
$refundResponse=json_decode($response);
```

Status Check

Step 1: Create TxnStatusRequest

```
$request=new TxnStatusRequest();
$request->setMerchantId("MERMER53c3jc3");
$request->setMerchantTxnId("1667980001729");
```

```
$request->setSignature("25a8c8feb0f4fc9bf0f0a8bd444314028f73c04ec202830271ddf359dc9cd6b4");
$request->setTxnReferenceId(null);
```

Step 2: Convert Request Object to Json (Used Php Json Extension)

```
$json=json_encode($request);
```

Step 3: Use curl_init() to initialize a new session and return a cURL handle.(Used cURL service in Php)

```
$curl = curl_init();
```

Step 4: Specify all headers required

```
curl_setopt_array($curl, array(
        CURLOPT_URL => 'https://spg-paymentorchestrator-qa.fcinternal.in/payment/v1/txn/status',
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_SSL_VERIFYPEER => false,
        CURLOPT_ENCODING => '',
        CURLOPT_MAXREDIRS => 10,
        CURLOPT_TIMEOUT => 0,
        CURLOPT_FOLLOWLOCATION => true,
        CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
        CURLOPT_CUSTOMREQUEST => 'POST',
        CURLOPT_POSTFIELDS =>$json,
        CURLOPT_HTTPHEADER => array(
                'Content-Type: application/json'
        ),
));
```

Step 5: Execute the request .

```
$response = curl_exec($curl);
```

Step 6: Deserialize the received response to a class object. (Use Php Json Extension

```
$initResponse=json_decode($response);
```